OFFERIGO · AI 求职工作台

AI 智能求职助手

技术文档

从简历素材到岗位匹配、定制文书、模拟面试与笔试训练的一站式 AI 求职平台

团队名称：待填写

团队成员：待填写

指导老师：待填写

文档版本：v1.0

提交日期：2026 年 11 月

目录

1. 项目概述
2. 需求分析
3. 系统架构设计
4. AI 技术方案
5. 核心功能实现
6. 创新点说明
7. 测试与部署
8. 总结与展望

1 项目概述

1.1 背景

高校毕业生规模持续增长，IT 行业岗位竞争激烈。求职者面临的困难并不只是"岗位数量不足"，而是三类结构性问题：其一，无法判断哪些岗位与自身经历真正匹配；其二，不擅长把零散、真实的经历组织成招聘方愿意阅读的简历与求职信；其三，面试与笔试缺少针对目标岗位的训练和反馈。

通用大模型虽然能生成文本，但直接用于求职场景存在明显风险：容易编造经历、同一份材料评分不稳定、建议无法追溯到事实。OfferiGo 的核心思路是**以用户自己的简历素材库作为唯一事实来源**，让 AI 完成检索、匹配、生成与演练，并对输出做确定性与可解释的约束。

1.2 项目目标

- 用一份可复用的素材库覆盖整个求职链路，避免重复填写。
- 提供稳定、可解释的岗位匹配评分，并明确"差在哪里、怎么补"。
- 按岗位生成定制简历与求职信，支持导出与迭代。
- 提供岗位定向的模拟面试与真题笔试训练，并沉淀错题与复盘。
- 把投递记录、简历归档和跟进提醒统一到个人中心。

1.3 项目范围

平台面向 IT 行业求职者，覆盖七大模块：简历素材库、岗位获取与收藏、匹配评估、定制文书、AI 模拟面试、AI 笔试助手、投递与跟进。不包含第三方招聘平台账号托管与自动代投，投递动作仍由用户在官方渠道完成，平台负责记录与跟进。

2 需求分析

2.1 用户画像

用户类型	典型诉求	平台价值
应届毕业生	经历零散、不会写简历、缺少面试经验	素材结构化、AI 定制文书、模拟面试与真题训练
转行求职者	经验与目标岗位不对口，需要证明可迁移能力	四维匹配评估、待补足项与补齐建议
在职跳槽者	时间有限，需要快速筛选岗位并高效准备	每日推荐、批量匹配、投递台账与跟进提醒

2.2 核心痛点与对应能力

痛点	现有做法的问题	本项目的方案

不知道适合什么岗位	凭感觉投递，命中率低	岗位要求逐条覆盖判定 + 四维量化评分，给出匹配优势与缺口
简历不会写、不会改	模板同质化，通用大模型容易编造	以素材为事实来源，检索相关素材后再生成，禁止编造
面试缺练习	缺少真实追问与反馈	岗位定向提问 + 实时语音识别 + 多维报告
笔试无针对性	通用题库与岗位无关	题库按岗位与技能匹配，支持错题归档与掌握标记
投递后无跟进	记录散落在表格与聊天里	投递台账、当次简历归档、状态与跟进提醒

2.3 功能需求（摘要）

模块	关键需求
简历素材库	多类型素材管理；导入简历；长文本 AI 拆分；增量补充；全站复用
岗位获取	每日自动爬取与推荐；粘贴/链接/文档/截图四种导入；收藏与分类筛选
匹配评估	确定性评分；匹配优势与待补足项；分数可追溯；高匹配提示
定制文书	模板选择；证件照；按岗位生成简历与求职信；PDF/Word 导出
AI 面试	岗位定向提问；实时语音；阶段反馈；报告与历史记录
AI 笔试	真题匹配；自由练习/限时模拟；组卷、批改、错题、掌握标记
投递跟进	投递记录；简历归档；状态维护；跟进提醒

2.4 非功能需求

- **稳定性**：AI 调用失败时必须有降级路径，不影响基础功能。
- **可解释性**：评分与待补足项可追溯到具体岗位要求与素材。
- **安全性**：密钥不落库不入仓；用户数据按账号隔离；外链抓取限制内网地址。
- **性能**：页面路由懒加载；重任务异步执行并展示真实阶段进度。
- **可用性**：亮暗双主题、移动端适配、键盘可达与减少动效支持。

3 系统架构设计

3.1 总体架构



3.2 模块划分

模块	职责	关键类
jobmatch	岗位库、素材库、匹配评估、简历与求职信导出、投递台账	JobMatchServiceImpl、JobApplicationService、JobMaterialSelector
jobrecommend	每日岗位爬取、向量推荐、推荐转目标岗位、历史 JD 补全	JobRecommendService、JobCrawlTask、各平台 Crawler
interview	面试会话编排、实时语音、报告生成	InterviewOrchestrator、ParaformerRealtimeClient
exam	题库抓取与匹配、组卷、批改、错题	ExamServiceImpl、ExamQuestionBankService、ExamQuestionCrawler
application	投递准备评估、定制简历、求职信、ATS 检查	ApplicationService
system	用户、鉴权、头像	SysUserServiceImpl、JwtInterceptor

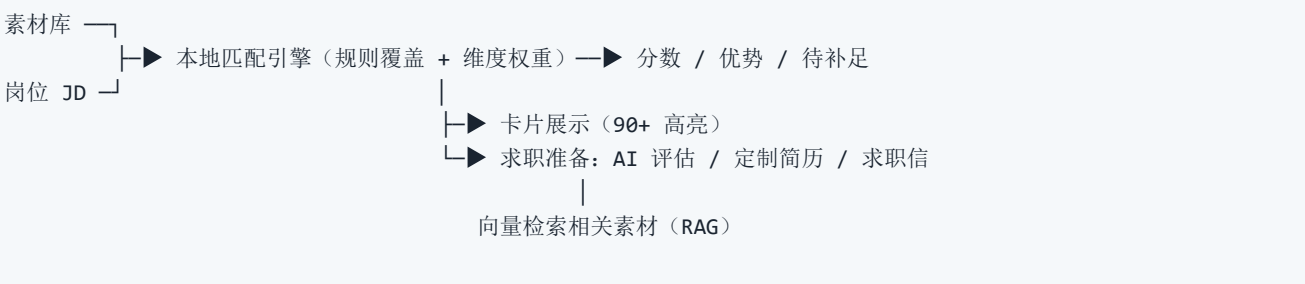
3.3 数据模型（核心表）

表	说明
sys_user	账号、密码哈希、头像
knowledge_entry	简历素材，按类型存储
target_job	用户收藏的岗位（目标岗位）
crawl_job	每日爬取的岗位池，含向量
job_match_result	匹配评估缓存与结果
interview_record	模拟面试记录与报告
exam_paper / wrong_question	笔试试卷与错题
job_application	投递台账，含归档简历路径

3.4 技术选型与理由

层次	选型	理由
前端框架	Vue 3 + Vite	组合式 API 便于拆分复杂交互；Vite 构建快，路由懒加载友好
UI	Element Plus + 自定义主题令牌	组件齐全，配合 CSS 变量实现亮暗双主题与统一设计语言
状态	Pinia	轻量、类型友好，适合用户与跨模块联动状态
后端	Spring Boot 3 + MyBatis-Plus	生态成熟，适合 REST + WebSocket 与定时任务
数据库	MySQL 8	事务与 JSON 字段满足素材、报告等半结构化数据
鉴权	JWT	无状态，前后端分离部署简单
抓取	Playwright	可渲染动态页面，适配多个招聘平台
文档导出	PDFBox / openhtmltopdf / POI	服务端生成，保证导出与预览一致

3.5 关键数据流：一次岗位匹配



大模型生成 → 结构化校验 → 导出

4 AI 技术方案

4.1 模型选型与理由

能力	模型/服务	选型理由
深度评估与文书	DeepSeek Chat	中文理解与结构化输出稳定，成本可控，适合长 JD 与多轮评估
通用对话与结构化提取	通义千问 qwen-plus	响应快、JSON 输出稳定，适合岗位信息标准化与批量任务
视觉识别	qwen-vl-plus	支持中文截图 OCR 与版面理解，用于 JD 截图导入
向量表示	text-embedding-v3	中文语义检索效果好，用于素材选择与岗位推荐
实时语音识别	Paraformer Realtime	WebSocket 流式识别，延迟低，支持中英混合

多模型分工的核心原则是"让合适的模型做合适的事"：结构化提取用低温度、强约束的模型；深度分析用推理能力更强的模型；实时性要求高的语音环节使用专用流式 ASR，而不是把音频交给通用大模型。

4.2 双轨评分体系

岗位匹配同时存在两套评分：

- **本地确定性引擎**：逐条判断岗位要求是否被素材覆盖（英文技术词、资格信号、中文短语与 bigram、经验年限数值比较），再按技能/经历/学历/契合四维加权。相同输入必然得到相同分数，且每条结论可追溯到具体素材。
- **后端大模型评估**：在用户主动"刷新评估"时调用，提供更细的分析与文案，但结果按"用户 + 岗位 + 素材内容"做缓存，内容不变时返回同一份结果。

这样既保证了列表与卡片的分数稳定可解释，又保留了深度分析能力；刷新不会出现"同一份简历分数乱跳"的问题。

4.3 基于个人素材的检索增强生成（RAG）

生成定制简历或求职信前，系统并不会把所有素材直接塞给模型，而是：

1. 将岗位名称、要求与技术栈拼接为查询文本，生成向量；
2. 对素材逐条生成向量并计算余弦相似度，选出最相关的条目；
3. 把筛选后的素材作为上下文交给大模型，并要求只使用这些事实；
4. 结构化解析输出，失败时回退原文，避免编造与格式损坏。

同一套向量能力也用于每日岗位推荐：用户技能向量与岗位向量计算相似度，作为推荐排序的基础。

4.4 多模态能力

- **图像**：岗位 JD 截图通过视觉模型识别为结构化岗位信息；证件照在前端完成裁剪与压缩后再上传。
- **语音**：模拟面试通过浏览器采集音频，经 WebSocket 送入实时 ASR，转写文本驱动追问与报告生成。
- **文本**：贯穿素材、岗位、评估、文书与题库。

4.5 阶段化 AI 工作流

简历、求职信、匹配评估、试卷生成等任务均通过统一的进度存储记录真实阶段（准备 → 生成 → 解析/保存），前端轮询并展示阶段、百分比与耗时。进度不是定时器伪造，而是由后端在关键节点写入，失败或完成时清理。

4.6 Prompt 设计与可控性

- 所有生成类 Prompt 明确"只使用给定素材，不编造经历、数据或技能"。
- 输出统一要求 JSON，服务端做结构化解析与字段校验；解析失败回退原文或明确的错误提示。
- 评估类 Prompt 拆分硬性门槛与软性维度，降低单次判断的随机性。
- 生成内容不直接覆盖用户原始素材，所有产物可预览、可修改。

4.7 优化策略

问题	策略
模型输出不稳定	低温度、结构化 JSON、字段校验、失败回退；评估结果按内容哈希缓存
重复调用成本	匹配结果与题库规模缓存；相同素材与岗位不重复评估
长任务阻塞	异步任务 + 阶段进度；前端轮询而非长连接阻塞
抓取并发冲突	爬虫串行锁 + 冷却时间；启动补跑与历史补全错开执行
外链安全	URL 合法性与内网地址校验，拒绝危险重定向目标

5 核心功能实现

5.1 简历素材库

素材按基础信息、教育经历、项目经历、实习经历、技能证书、荣誉奖项等类型存储。支持粘贴长文本后由模型拆分为多条结构化记录，也支持导入简历文件并做 OCR/解析。素材是后续所有 AI 生成的唯一事实来源，修改后相关评估与生成结果按版本失效。

5.2 岗位获取与收藏

每日 08:00 通过 Playwright 抓取智联、腾讯、字节等公开岗位，按用户技能向量排序生成 TOP10 推荐；同时支持粘贴、链接、文档、截图四种导入方式。用户"收藏"的岗位即为目标岗位池，也是匹配评估的对象。爬虫对缺失的岗位要求做历史补全，避免只有摘要。

5.3 匹配评估

本地引擎对岗位要求逐条判定，输出匹配优势与待补足项，并按四维权重汇总为总分；分数 90 以上的岗位在列表中以左侧强调条、渐变分数与"高匹配"标签突出显示。用户可在求职准备中查看 AI 深度评估，评估结果稳定且可缓存。

5.4 定制文书

用户选择模板、上传证件照后，系统按岗位检索相关素材并生成定制简历与求职信。前端提供预览，服务端负责导出 PDF/Word，保证预览与导出效果一致。生成过程有真实阶段进度，可随时重试。

5.5 AI 模拟面试

面试编排器根据岗位 JD 生成问题序列，实时语音识别用户回答并判断是否需要追问。结束后生成包含多维度评分的报告，历史记录可回看。语音链路使用 WebSocket，鉴权沿用 JWT。

5.6 AI 笔试助手

题库来自公开面经与题目合集，按岗位、技能标签与难度匹配真题；支持自由练习与限时模拟。组卷、补全解析、批改均有阶段进度；错题自动归档，可重做与标记掌握。题库每日 08:00 更新，只有真正新增题目时才提示更新。

5.7 投递与跟进

投递记录进入个人中心台账，支持上传并归档当次投递的简历文件、维护投递状态、设置跟进提醒。删除记录时同步清理归档文件，避免磁盘残留。

6 创新点说明

6.1 技术创新点

1. **双轨评分 + 内容哈希缓存**：确定性引擎负责稳定与可解释，大模型负责深度分析；按素材内容缓存，解决"刷新分数漂移"这类生成式应用的常见问题。
2. **面向个人素材的 RAG**：不是通用知识检索，而是在用户自己的素材库中做向量筛选，生成内容天然可追溯、低编造。
3. **多模态求职链路**：文本、截图、语音分别使用合适的模型能力，覆盖"看岗位、答面试"的真实场景。
4. **真实阶段化工作流**：生成任务的状态由后端写入，前端展示真实进度，失败可恢复。

6.2 应用创新点

1. 把"简历素材"做成一次录入、全链路复用的基础设施，而不是每个功能各写一份。
2. 把岗位匹配、文书生成、面试演练、笔试训练和投递跟进串成闭环，用户不需要在多个工具之间迁移。
3. 面向毕业生的就业公平价值：用低成本的 AI 辅导替代部分付费简历/面试服务。

7 测试与部署

7.1 测试

- 后端包含进度存储、URL 安全校验、向量计算、导出器、代码沙箱等单元测试。
- 关键流程（登录、素材、匹配、生成、面试、笔试、投递）在开发过程中进行手工回归。
- 前端构建与后端编译作为提交前的基线检查。

7.2 部署

- 后端打包为可执行 JAR，通过环境变量注入数据库与模型密钥。
- 前端构建为静态资源，由 Nginx 托管并反向代理 `/api` 与 `/ws`。
- 提供 Docker Compose 示例（MySQL + 后端 + 前端）用于在线演示。
- 数据库表由后端启动时自动初始化，便于评审环境快速搭建。

7.3 安全

- 密钥仅来自环境变量或本地未提交配置；公开仓库不含真实 Key。
- JWT 鉴权；用户数据按账号隔离。
- 外链抓取校验 URL 与内网地址；上传文件校验类型与大小。
- AI 生成禁止编造，解析失败有回退。

8 总结与展望

OfferiGo 用一份可复用的简历素材库和一套可控的 AI 工作流，把求职过程中最耗时的匹配、改写、演练与跟进串成闭环。项目的重点不是"让 AI 写更多字"，而是让 AI 在真实约束下做更准确、更可解释的事。

后续计划：

- 引入流式输出，让评估与文书生成过程实时可见。
- 通过 Function Calling / MCP 把岗位搜索、题库检索等能力封装为工具，让模型参与规划与调用。
- 前端逐步引入 TypeScript 并拆分超大组件，提升可维护性。
- 补充端到端测试与性能监控，支撑更大规模使用。